

C++ STUDY NOTES

CHAPTER 1

Introduction to C++

OOP Concepts, Data Types, Pointers, References, Strings, Arrays, Enums, const & volatile Qualifiers, typedef, and an Introduction to Classes

University : Rani Channamma University, Belagavi
Course Code : 24MSCCS 1.3
Programme : M.Sc. Computer Science
Semester : First Semester
Syllabus : 2024-25 onwards
Author : Sudarshan Badli
Academic Yr : 2026-2027

AUTHORSHIP

Prepared and Created by: Sudarshan Badli
© 2026 Sudarshan Badli. All Rights Reserved.

Table of Contents

1.1 OOP Concepts	3
1.2 C++ Data Types	4
1.3 Literal Constants	5
1.4 The bool Data Type	6
1.5 Variables — Declaration and Naming	6
1.6 Definition of an Object	7
1.7 Pointer Types	8
1.8 String Types	9
1.9 const Qualifier	10
1.10 Reference Types	11
1.11 Enumeration Types	12
1.12 Array Types and Multidimensional Arrays	13
1.13 Relationship of Array and Pointer Types	14
1.14 typedef Names	15
1.15 volatile Qualifier	16
1.16 Class Types — a First Look	17
1.22 Examination Preparation — Chapter 1	20

HOW THIS CHAPTER IS ORGANISED

- Follows the official RCUB M.Sc. Computer Science syllabus (24MSCCS 1.3), Unit I, exactly — no topic is skipped or compressed.
- Each topic: Definition → Syntax → Key points → Diagram/Table (where relevant) → Full worked C++ program(s) with output.
- Sections 1.17–1.21 add deep-dive material: dry runs/trace tables, line-by-line explanation, advantages/disadvantages/applications, common mistakes, and head-to-head comparisons.
- Section 1.22 is a complete examination-preparation pack: 2/5/10-mark Q&A; programming questions, output prediction, find-the-error, MCQs and viva questions.

1.1 OOP Concepts

DEFINITION

Object-Oriented Programming (OOP) organises software around **objects** — entities that bundle data (attributes) and the functions (behaviour) that act on that data — rather than around a sequence of instructions. This contrasts with procedural programming (e.g. plain C), where data and functions are separate.

KEY POINTS

- **Class**: a user-defined blueprint/type that specifies the data members and member functions an object of that type will have.
- **Object**: a concrete instance of a class, occupying memory, with its own copy of the data members.
- **Encapsulation**: binding data and the functions that operate on it into a single unit (class), and hiding internal details using access specifiers (private/protected) — also called data hiding.
- **Abstraction**: exposing only essential features of an object while hiding implementation complexity (e.g. you drive a car without knowing engine internals).
- **Inheritance**: a mechanism by which a new class (derived) acquires the properties and behaviour of an existing class (base), supporting code reuse.
- **Polymorphism**: "many forms" — the same function name or operator behaves differently depending on the object/context (function overloading, operator overloading, virtual functions).
- **Message passing**: objects communicate by invoking each other's member functions.

PROGRAM – ILLUSTRATING ENCAPSULATION

```
#include <iostream>
using namespace std;

class Account {
private:                // hidden data (encapsulation)
    double balance;
public:
    Account(double b) : balance(b) {}
    void deposit(double amt) { balance += amt; }
    double getBalance() { return balance; } // controlled access
};

int main() {
    Account a(1000);
    a.deposit(500);
    cout << "Balance = " << a.getBalance() << "\n";
    return 0;
}
```

OUTPUT

```
Balance = 1500
```

KEY POINTS

- OOP's four pillars asked frequently in exams: **Encapsulation, Abstraction, Inheritance, Polymorphism** (EAIP).
- Procedural programming: top-down, functions act on global/shared data. OOP: bottom-up, data and functions are bound inside objects.

1.
2

C++ Data Types

DEFINITION

A **data type** tells the compiler the kind of value a variable holds and how much memory to reserve for it. C++ data types are grouped into *fundamental (built-in)*, *derived* and *user-defined* types.

Type	Typical Size	Range (typical)
char	1 byte	-128 to 127
int	4 bytes	-2,147,483,648 to 2,147,483,647
float	4 bytes	$\pm 3.4\text{e}-38$ to $\pm 3.4\text{e}38$ (6-7 digit precision)
double	8 bytes	$\pm 1.7\text{e}-308$ to $\pm 1.7\text{e}308$ (15 digit precision)
bool	1 byte	true (1) or false (0)

Example

```
#include <iostream>
using namespace std;

int main() {
    cout << "int      : " << sizeof(int) << " bytes\n";
    cout << "float    : " << sizeof(float) << " bytes\n";
    cout << "double   : " << sizeof(double) << " bytes\n";
    cout << "char     : " << sizeof(char) << " bytes\n";
    cout << "bool    : " << sizeof(bool) << " bytes\n";
    return 0;
}
```

OUTPUT

```
int      : 4 bytes
float    : 4 bytes
double   : 8 bytes
char     : 1 byte
bool     : 1 byte
```

1.
3

Literal Constants

DEFINITION

A **literal** is a fixed value written directly in source code (not stored in a variable that can change). C++ has integer, floating-point, character, string and boolean literals.

SYNTAX

```
int    a = 42;           // decimal integer literal
int    b = 052;          // octal (leading 0)
int    c = 0x2A;          // hexadecimal (leading 0x)
float  f = 3.14f;         // floating literal (f suffix = float)
double d = 3.14;          // double by default
char   ch = 'A';          // character literal
string s = "Hello";       // string literal
bool   ok = true;         // boolean literal
```

KEY POINTS

- Integer literals can be written in decimal, octal (prefix 0) or hexadecimal (prefix 0x/0X).
- A floating literal defaults to type `double`; append `f/F` for float, `l/L` for long double.
- A character literal is enclosed in single quotes and occupies 1 byte; a string literal is enclosed in double quotes and is a null-terminated array of characters (or a `std::string` object).

Escape sequences

Escape sequences represent characters that are hard to type directly (control characters, quotes, backslash). They begin with a backslash.

Sequence	Meaning	Sequence	Meaning
<code>\n</code>	New line	<code>\t</code>	Horizontal tab
<code>\\</code>	Backslash	<code>\'</code>	Single quote
<code>\"</code>	Double quote	<code>\0</code>	Null character
<code>\b</code>	Backspace	<code>\r</code>	Carriage return
<code>\a</code>	Alert (bell)	<code>\f</code>	Form feed

```
#include <iostream>
using namespace std;

int main() {
    cout << "Name:\tAsha\nCity:\tBelagavi\n";
    cout << "She said, \"C++ is fun\"\n";
    cout << "Path: C:\\Users\\Asha\n";
    return 0;
}
```

OUTPUT

```
Name:█Asha
City:█Belagavi
She said, "C++ is fun"
Path: C:\Users\Asha
```

1.4 The bool Data Type

DEFINITION

The **bool** type stores a logical (Boolean) value: either **true** (1) or **false** (0). It is produced naturally by relational (<, >, ==) and logical (&&, ||, !) operators, and used to control if/loop conditions.

SYNTAX

```
bool flag = true;
bool isEven = (n % 2 == 0);
```

KEY POINTS

- Any non-zero value converts to **true**; zero converts to **false**.
- cout << boolalpha prints **true/false** as words instead of 1/0.

PROGRAM

```
#include <iostream>
using namespace std;

int main() {
    bool isPrime = true;
    int n = 15;
    for (int i = 2; i < n; i++)
        if (n % i == 0) { isPrime = false; break; }
    cout << boolalpha << n << " prime? " << isPrime << "\n";
    return 0;
}
```

OUTPUT

```
15 prime? false
```

1.5 Variables — Declaration and Naming

DEFINITION

A **variable** is a named memory location that can hold a value of a given type, and whose value may change during execution. It must be declared with a type before use.

SYNTAX

```
type variableName;
type variableName = initialValue;
type v1, v2, v3;           // multiple variables
```

KEY POINTS

- Naming rules: letters, digits and underscore only; cannot start with a digit; case-sensitive; cannot be a C++ keyword.
- C++ has 90+ reserved keywords, e.g. **int**, **class**, **public**, **private**, **virtual**, **static**, **const**, **new**, **delete**, **try**, **catch**, **template**.
- Good practice: use meaningful, camelCase names (**totalMarks**, not **tm**).
- A variable must be **defined** (given storage) and may be separately **declared** (e.g. with **extern**) in another file.

PROGRAM

```
#include <iostream>
using namespace std;

int main() {
    int rollNo = 101;
    float marks = 88.5f;
    char grade = 'A';
    const double PI = 3.14159; // cannot be changed later
    cout << rollNo << " " << marks << " " << grade << " " << PI << "\n";
    return 0;
}
```

OUTPUT

```
101 88.5 A 3.14159
```

1. Definition of an Object

DEFINITION

An **object** is a region of storage with a type, created either explicitly from a class (Student s1;) or implicitly as any variable of a fundamental type. In OOP, "object" usually refers to an instance of a class, carrying its own data members and able to invoke the class's member functions.

SYNTAX

```
ClassName objectName; // default construction
ClassName objectName(arguments); // parameterised construction
```

KEY POINTS

- Each object has its own copy of the non-static data members, but shares the same member-function code.
- Objects have a lifetime (scope: local, global, dynamic) after which they are destroyed.

PROGRAM

```
#include <iostream>
using namespace std;

class Point {
public:
    int x, y;
    void show() { cout << "(" << x << "," << y << ")\n"; }
};

int main() {
    Point p1; // object 1
    p1.x = 2; p1.y = 3;
    Point p2 = {5, 7}; // object 2, aggregate init
    p1.show(); p2.show();
    return 0;
}
```

OUTPUT

```
(2,3)
(5,7)
```

1.7

Pointer Types

DEFINITION

A **pointer** is a variable that stores the memory address of another variable, rather than a value directly. Pointers enable dynamic memory management, efficient array/string handling, and passing large objects by reference.

SYNTAX

```
type *pointerName;           // declaration
pointerName = &variable;    // store address
*pointerName                 // dereference: access the pointed-to value
```

KEY POINTS

- & is the *address-of* operator; * is the *dereference* operator (also used in the declaration to mark a pointer type).
- A void* pointer can hold the address of any type but must be cast before dereferencing.
- An uninitialised ("wild") pointer must not be dereferenced; set unused pointers to nullptr.
- Pointer arithmetic (p+1) moves by sizeof(type) bytes, used heavily with arrays.

PROGRAM – BASIC POINTER

```
#include <iostream>
using namespace std;

int main() {
    int x = 25;
    int *p = &x;           // p holds address of x
    cout << "Value of x      : " << x << "\n";
    cout << "Address of x    : " << &x << "\n";
    cout << "Value stored in p: " << p << "\n";
    cout << "Value pointed by p: " << *p << "\n";
    *p = 50;                // modify x through the pointer
    cout << "x after *p=50   : " << x << "\n";
    return 0;
}
```

OUTPUT

```
Value of x      : 25
Address of x    : 0x61fe14
Value stored in p: 0x61fe14
Value pointed by p: 25
x after *p=50   : 50
```

PROGRAM – DYNAMIC MEMORY (NEW / DELETE)

```
#include <iostream>
using namespace std;

int main() {
    int *p = new int(99); // allocate on heap
    cout << "Heap value = " << *p << "\n";
    delete p;             // release memory
    p = nullptr;
    return 0;
}
```

OUTPUT

```
Heap value = 99
```

KEY POINTS

- Addresses printed will differ each run; only the pattern (a hex-looking value) matters in an exam answer.

1. String Types

DEFINITION

C++ supports two ways to represent text: the C-style **character array** (null-terminated `char []`) inherited from C, and the C++ Standard Library **`std::string`** class, which manages its own memory and offers convenient operators and member functions.

SYNTAX

```
// C-style string
char name[20] = "Asha";

// C++ string class
#include <string>
string name = "Asha";
```

KEY POINTS

- A C-string always ends with a null character `'\0'`, added automatically for string literals.
- `std::string` supports `+` for concatenation, `==` for comparison, `.length()`, `.substr()`, `[]` indexing, and grows dynamically.
- Prefer `std::string` in modern C++; C-strings are mainly seen in legacy code and low-level APIs.

PROGRAM

```
#include <iostream>
#include <cstring>
#include <string>
using namespace std;

int main() {
    char cname[20] = "Asha";           // C-style string
    cout << "C-string: " << cname << ", length = " << strlen(cname) << "\n";

    string name = "Asha";              // std::string
    string full = name + " Patil";      // concatenation
    cout << "std::string: " << full << ", length = " << full.length() << "\n";
    cout << "Substring   : " << full.substr(0, 4) << "\n";
    return 0;
}
```

OUTPUT

```
C-string: Asha, length = 4
std::string: Asha Patil, length = 10
Substring   : Asha
```

1.9

const Qualifier

DEFINITION

The **const** keyword marks a variable, pointer, function parameter, or member function as unmodifiable. Attempting to change a const value is a compile-time error, which helps prevent accidental modification.

SYNTAX

```
const type name = value;           // constant variable
const type *p;                     // pointer to constant data
type *const p;                     // constant pointer
returnType fn(...) const;          // const member function (unit II)
```

KEY POINTS

- A const variable must be initialised at the point of declaration.
- "const int *p" (pointer to const) — the data cannot change through p. "int *const p" (const pointer) — p itself cannot be re-pointed.
- A const member function promises not to modify the object's data members (covered under Classes).

PROGRAM

```
#include <iostream>
using namespace std;

int main() {
    const double PI = 3.14159;
    int x = 10, y = 20;
    const int *p1 = &x;           // *p1 cannot change
    int *const p2 = &x;           // p2 cannot be re-pointed
    // *p1 = 5;    error          p1 = &y;    OK
    *p2 = 5;        // OK          // p2 = &y;    error
    cout << "PI = " << PI << ", x via p2 = " << x << "\n";
    return 0;
}
```

OUTPUT

```
PI = 3.14159, x via p2 = 5
```

1.10 Reference Types

DEFINITION

A **reference** is an alias — another name — for an already-existing variable. Unlike a pointer, a reference must be initialised when declared, can never be made to refer to a different variable, and is used directly (no dereference operator needed).

SYNTAX

```
type &referenceName = existingVariable;

// common use: pass-by-reference parameter
void fn(type &param) { ... }
```

KEY POINTS

- A reference shares the same memory address as the variable it refers to; modifying one modifies the other.
- References cannot be null and cannot be reseated (bound to a different variable later) — this is the key difference from a pointer.
- Pass-by-reference avoids copying large objects and allows a function to modify the caller's variable.

PROGRAM – REFERENCE ALIAS

```
#include <iostream>
using namespace std;

int main() {
    int a = 10;
    int &ref = a;      // ref is another name for a
    ref = 20;          // changes a as well
    cout << "a = " << a << ", ref = " << ref << "\n";
    return 0;
}
```

OUTPUT

```
a = 20, ref = 20
```

PROGRAM – SWAP USING REFERENCE PARAMETERS

```
#include <iostream>
using namespace std;

void swapNums(int &x, int &y) {
    int t = x; x = y; y = t;
}

int main() {
    int p = 3, q = 8;
    swapNums(p, q);
    cout << "p = " << p << ", q = " << q << "\n";
    return 0;
}
```

OUTPUT

```
p = 8, q = 3
```

1.
11

Enumeration Types

DEFINITION

An **enum** defines a user-named type consisting of a set of named integer constants, making code more readable than using plain numbers (magic numbers).

SYNTAX

```
enum Day { SUN, MON, TUE, WED, THU, FRI, SAT };    // SUN=0 ... SAT=6
enum Grade { A = 5, B = 4, C = 3 };               // custom starting values

// scoped (C++11), avoids name clashes
enum class Color { RED, GREEN, BLUE };
```

KEY POINTS

- By default, enumerators start at 0 and increase by 1; any enumerator can be given an explicit value, and later ones continue from it.
- A plain enum value implicitly converts to `int`; an `enum class` (scoped enum) does not, which is safer.

PROGRAM

```
#include <iostream>
using namespace std;

enum Day { SUN, MON, TUE, WED, THU, FRI, SAT };

int main() {
    Day today = WED;
    cout << "Day code = " << today << "\n";
    if (today == SAT || today == SUN)
        cout << "Weekend\n";
    else
        cout << "Weekday\n";
    return 0;
}
```

OUTPUT

```
Day code = 3
Weekday
```

1.
12

Array Types and Multidimensional Arrays

DEFINITION

An **array** is a fixed-size collection of elements of the same type, stored in contiguous memory and accessed using an index starting at 0. A **multidimensional array** (e.g. a matrix) is an array of arrays.

SYNTAX

```
type arr[size];           // 1-D array
type arr[size] = {v1, v2, ..., vn}; // with initializer
type mat[rows][cols];    // 2-D array
```

KEY POINTS

- Array indices run from 0 to size-1; accessing outside this range is undefined behaviour (no automatic bounds checking).
- The array name decays to a pointer to its first element in most expressions.
- A 2-D array `mat[r][c]` is stored row-by-row (row-major order) in memory.

PROGRAM – 1-D ARRAY – SUM AND AVERAGE

```
#include <iostream>
using namespace std;

int main() {
    int marks[5] = {70, 85, 90, 60, 75};
    int sum = 0;
    for (int i = 0; i < 5; i++) sum += marks[i];
    cout << "Sum = " << sum << ", Average = " << (double)sum / 5 << "\n";
    return 0;
}
```

OUTPUT

```
Sum = 380, Average = 76
```

PROGRAM – 2-D ARRAY – MATRIX ADDITION

```
#include <iostream>
using namespace std;

int main() {
    int a[2][2] = {{1,2},{3,4}};
    int b[2][2] = {{5,6},{7,8}};
    int c[2][2];
    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 2; j++)
            c[i][j] = a[i][j] + b[i][j];
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) cout << c[i][j] << " ";
        cout << "\n";
    }
    return 0;
}
```

OUTPUT

```
6 8
10 12
```

1.
13

Relationship of Array and Pointer Types

DEFINITION

An array name, when used in most expressions, **decays** into a pointer to its first element. This means array elements can be accessed through pointer arithmetic, and the two notations `arr[i]` and `*(arr + i)` are equivalent.

SYNTAX

```
int arr[5] = {10, 20, 30, 40, 50};
int *p = arr;           // same as &arr[0]
p[2] == *(p + 2) == arr[2]
*(arr + 2) == arr[2]
```

KEY POINTS

- `sizeof(arr)` gives the whole array's size in bytes, but `sizeof(p)` (a pointer) gives only the pointer's size — they are not interchangeable for sizing.
- Passing an array to a function passes a pointer to its first element (size information is lost unless passed separately).

PROGRAM

```
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int *p = arr;
    for (int i = 0; i < 5; i++)
        cout << "arr[" << i << "]=" << arr[i]
              << " *(p+" << i << ")=" << *(p + i) << "\n";
    return 0;
}
```

OUTPUT

```
arr[0]=10 *(p+0)=10
arr[1]=20 *(p+1)=20
arr[2]=30 *(p+2)=30
arr[3]=40 *(p+3)=40
arr[4]=50 *(p+4)=50
```

1.14

typedef Names

DEFINITION

The **typedef** keyword creates an alias (synonym) for an existing type, making complex declarations shorter and more readable. The modern equivalent in C++11 is **using**.

SYNTAX

```
typedef existingType aliasName;  
  
// C++11 alternative  
using aliasName = existingType;
```

KEY POINTS

- A typedef does not create a new type, only a new name for an existing one; the compiler treats them identically.
- Commonly used for long template names, function pointers and portability (e.g. a fixed-width integer).

PROGRAM

```
#include <iostream>  
using namespace std;  
  
typedef unsigned long ulong;  
using Marks = double;           // C++11 style  
  
int main() {  
    ulong population = 1400000000UL;  
    Marks avg = 86.5;  
    cout << population << " " << avg << "\n";  
    return 0;  
}
```

OUTPUT

```
1400000000 86.5
```

1.15

volatile Qualifier

DEFINITION

The **volatile** keyword tells the compiler that a variable's value may change at any time outside the normal flow of the program (e.g. modified by hardware, an interrupt service routine, or another thread). It prevents the compiler from applying optimisations that assume the value stays constant between accesses (such as caching it in a register).

SYNTAX

```
volatile type variableName;  
volatile int flag;  
const volatile int sensorReading;    // both qualifiers can combine
```

KEY POINTS

- Every read of a volatile variable is fetched freshly from memory instead of from a cached/optimised copy.
- Typical use: memory-mapped hardware registers, flags set inside interrupt handlers, variables shared with another thread (though `std::atomic` is preferred for thread-safety in modern C++).

PROGRAM

```
#include <iostream>  
using namespace std;  
  
int main() {  
    volatile int sensor;    // value may change externally  
    sensor = 0;  
    // In real embedded code, an interrupt handler could  
    // update 'sensor' asynchronously; the compiler will  
    // not optimise away repeated reads of it.  
    cout << "Initial sensor value = " << sensor << "\n";  
    return 0;  
}
```

OUTPUT

```
Initial sensor value = 0
```

1.16 Class Types — a First Look

DEFINITION

A **class** is C++'s mechanism for user-defined types: it groups related data members and member functions under one name, with access control. A class is a *type*; an object is a *variable* of that type. Full coverage (constructors, destructors, access specifiers, this pointer, static members, operator overloading) is in Unit II.

SYNTAX

```
class ClassName {
private:
    dataType member1;
public:
    returnType function1(parameters);
};

ClassName obj;           // create an object
obj.function1(args);     // call via dot operator
```

KEY POINTS

- By default, members of a `class` are `private`; members of a `struct` are `public` — that is the only structural difference between the two in C++.
- Member functions are accessed using the dot operator (`.`) on an object, or the arrow operator (`->`) through a pointer to an object.

PROGRAM

```
#include <iostream>
using namespace std;

class Rectangle {
private:
    double length, breadth;
public:
    void setDimensions(double l, double b) { length = l; breadth = b; }
    double area() { return length * breadth; }
};

int main() {
    Rectangle r;
    r.setDimensions(5.0, 3.0);
    cout << "Area = " << r.area() << "\n";
    return 0;
}
```

OUTPUT

```
Area = 15
```

KEY POINTS

- See Unit II for constructors, destructors, friend functions, static members and operator overloading.

1.17 Deep Dive — Dry Run of the `sizeof()` Program (Topic 1.2)

TRACE TABLE

Statement executed	Expression evaluated	Output produced
<code>cout << sizeof(int)</code>	Compiler substitutes the constant 4 (bytes) on a typical 64-bit system	"int : 4 bytes"
<code>cout << sizeof(float)</code>	Compiler substitutes 4	"float : 4 bytes"

Statement executed	Expression evaluated	Output produced
cout << sizeof(double)	Compiler substitutes 8	"double : 8 bytes"
cout << sizeof(char)	Compiler substitutes 1	"char : 1 byte"
cout << sizeof(bool)	Compiler substitutes 1	"bool : 1 byte"

sizeof is evaluated entirely at compile time (it is an operator, not a function call), so there is no runtime computation — the trace table above shows substitution, not execution steps.

1.18 Deep Dive — Pointers: Advantages, Disadvantages, Applications

Advantages, Disadvantages and Applications

ADVANTAGES

- Enable dynamic memory allocation (heap objects created and sized at run time).
- Allow a function to modify the caller's variables (pass-by-address).
- Make array traversal and manipulation efficient via pointer arithmetic.
- Essential for building linked data structures (linked lists, trees, graphs).
- Allow passing large structures/objects to functions without copying them.

DISADVANTAGES

- Dereferencing an uninitialised or NULL pointer causes undefined behaviour / crashes.
- Manual memory management (new/delete) can cause memory leaks if delete is forgotten.
- Pointer arithmetic errors can silently corrupt unrelated memory.
- Code using raw pointers heavily is harder to read and debug than reference- or container-based code.

APPLICATIONS

- Dynamic data structures: linked lists, stacks, queues, trees, graphs.
- Dynamic memory allocation for arrays/objects whose size is known only at run time.
- Implementing call-by-reference-like behaviour in C-style functions.
- Low-level systems programming: device drivers, embedded systems, memory-mapped I/O.

Line-by-Line Explanation

Line / Statement	Explanation
int x = 25;	Declares an int variable x and initialises it with value 25 in a normal (stack) memory location.
int *p = &x;	Declares p as a pointer to int, then stores the address of x in p using the address-of operator &.
cout << x;	Prints the value stored in x directly: 25.
cout << &x;	Prints the memory address of x (an implementation-specific hexadecimal value).
cout << p;	Prints the value stored in p, which is the address of x — identical to &x.
cout << *p;	Dereferences p (follows the pointer) to access the value stored at that address: 25.
*p = 50;	Dereferences p and assigns 50 to the memory location it points to — this changes x itself, since p points to x.

COMMON MISTAKES / DEBUGGING TIPS

- Dereferencing a pointer before assigning it an address (a "wild" or "dangling" pointer) — always initialise pointers, or set them to nullptr.

- Confusing `*p = 5;` (assign through the pointer, changes the pointee) with `p = &y;` (re-point `p` itself, does not change the old pointee).
- Forgetting `delete` (or `delete[]` for arrays) after `new`, causing a memory leak.
- Using a pointer after it has been deleted (a "dangling pointer") without setting it to `nullptr`.
- Mixing up `delete p;` (single object) with `delete[] p;` (array) — using the wrong form is undefined behaviour.

1.19 Deep Dive — Arrays: Advantages, Disadvantages, Applications

Advantages, Disadvantages and Applications

ADVANTAGES

- Store multiple values of the same type under one name, with $O(1)$ access by index.
- Contiguous memory layout gives excellent cache performance for sequential processing.
- Simple, predictable syntax; the foundation for strings, matrices, and many algorithms.

DISADVANTAGES

- Fixed size decided at compile time (for plain C-style arrays) — cannot grow or shrink at run time.
- No automatic bounds checking: reading/writing outside the valid index range is undefined behaviour.
- Inserting or deleting an element in the middle requires manually shifting all subsequent elements.

APPLICATIONS

- Storing fixed-size collections: student marks, sensor readings, lookup tables.
- Implementing matrices and other multidimensional numerical data.
- Underlying storage for higher-level containers such as `std::vector` (Unit V).

DRY RUN — MATRIX ADDITION PROGRAM (TOPIC 1.12)

i	j	a[i][j]	b[i][j]	c[i][j] = a[i][j]+b[i][j]
0	0	1	5	6
0	1	2	6	8
1	0	3	7	10
1	1	4	8	12

COMMON MISTAKES / DEBUGGING TIPS

- Off-by-one errors: using `i <= size` instead of `i < size`, which reads/writes one element past the array.
- Assuming `sizeof(arr)` still works after the array has decayed to a pointer (e.g. inside a function parameter) — it then returns the pointer's size, not the array's.
- Forgetting that 2-D array indices are `[row][col]`, not `[col][row]`, when the matrix is not square.

1.20 Comparison — Pointer vs Reference (frequently asked)

Point	Pointer	Reference
Null value	Can be NULL / <code>nullptr</code>	Cannot be null — must refer to a valid object
Reseating	Can be made to point elsewhere	Cannot be reseated once initialised
Initialisation	Can be declared without initialising	Must be initialised at declaration
Syntax to access value	Needs dereference operator <code>*</code>	Used directly, like the original variable
Arithmetic	Supports pointer arithmetic (<code>p+1</code>)	No arithmetic on references
Memory	Occupies its own memory (stores an address)	Typically no separate storage; just an alias

1.21 Comparison — C-style String vs std::string (frequently asked)

Point	char array (C-string)	std::string
Header	none required (built-in) /	#include
Size	Fixed at declaration; must leave room for \0	Grows and shrinks dynamically
Concatenation	Needs strcat() / manual loop	Direct + operator
Comparison	strcmp() (returns 0 if equal)	Direct == operator
Safety	No bounds checking; buffer overflow risk	Managed internally, safer
Termination	Always null-terminated (\0)	Keeps its own length; no \0 requirement

1.
22

Examination Preparation — Chapter 1

NOTE ON QUESTION SOURCE

- The questions below are **expected / practice questions** prepared from the official RCUB syllabus for this chapter, written in the common style of university exam papers.
- They are **not** claimed to be verified previous-year question papers — no actual RCUB past paper was available to copy from. Cross-check against your department's official previous-year papers for the exact verified questions, if available.

A. Two-Mark Questions and Answers

Q1. [2 marks] Define a class and an object.

A class is a user-defined blueprint/type that groups data members and member functions. An object is a concrete instance of a class, occupying memory with its own copy of the data members.

Q2. [2 marks] What is encapsulation?

Encapsulation is the binding of data and the functions that operate on it into a single unit (class), combined with hiding internal details using access specifiers such as private.

Q3. [2 marks] Differentiate between a pointer and a reference in one line each.

A pointer stores an address and can be reseated or left null; a reference is a permanent alias for an existing variable, must be initialised, and cannot be reseated or null.

Q4. [2 marks] What is the purpose of the const qualifier?

const marks a variable, pointer, or function parameter as unmodifiable, so the compiler flags any attempt to change its value, preventing accidental modification.

Q5. [2 marks] What is type casting? Give an example.

Type casting is explicitly converting a value from one data type to another, e.g. (double)sum / count, to force floating-point division.

Q6. [2 marks] What is the default value of the first enumerator in an unscoped enum?

Zero (0); each subsequent enumerator is one more than the previous, unless given an explicit value.

Q7. [2 marks] Why must a reference be initialised at the point of declaration?

Because a reference is only an alias for an existing variable, not a separate object, so it must be bound to something at creation; C++ provides no way to leave it unbound.

Q8. [2 marks] What is the difference between declaration and definition of a variable?

A declaration tells the compiler a variable's type and name (may use extern, no storage allocated); a definition allocates actual storage for the variable.

Q9. [2 marks] What does the escape sequence \n do?

It inserts a new line character, moving the cursor to the beginning of the next line in the output.

Q10. [2 marks] What is a null-terminated string?

A C-style character array that ends with the special null character `\0`, marking where the string's content stops.

Q11. [2 marks] What is the use of the `sizeof` operator?

`sizeof` returns the number of bytes occupied in memory by a type or a variable, evaluated at compile time.

Q12. [2 marks] What is a wild (uninitialised) pointer?

A pointer that has been declared but not yet assigned a valid address; dereferencing it causes undefined behaviour.

Q13. [2 marks] What is the difference between `=` and `==` in C++?

`=` is the assignment operator (stores a value into a variable); `==` is the equality relational operator (compares two values and yields true/false).

Q14. [2 marks] State one advantage of `std::string` over a char array.

`std::string` grows dynamically and manages its own memory, avoiding manual buffer-size management and the risk of buffer overflow.

Q15. [2 marks] What is the volatile qualifier used for?

It tells the compiler a variable's value may change outside normal program flow (e.g. hardware, interrupt), so the compiler must not optimise away repeated reads of it.

B. Five-Mark Questions and Answers

Q1. [5 marks] Explain the four pillars of OOP with a suitable example for each.

Encapsulation binds data and functions together and hides internal state behind private access, exposing only public getter/setter methods (e.g. an Account class hiding its balance). Abstraction exposes only essential features while hiding implementation complexity (e.g. calling `area()` on a Shape without knowing the formula used internally). Inheritance lets a derived class acquire the members of a base class, enabling code reuse (e.g. `class Dog : public Animal`). Polymorphism allows the same function name/operator to behave differently depending on context, through function overloading, operator overloading or virtual functions (e.g. `add(int,int)` vs `add(double,double)`). Together these four principles make OOP code modular, reusable and easier to maintain than purely procedural code.

Q2. [5 marks] Differentiate between C-style strings and `std::string`, with an example program.

A C-style string is a null-terminated char array with a fixed size decided at declaration, concatenated/compared using functions like `strcat()` and `strcmp()`, and with no automatic bounds checking (risk of buffer overflow). `std::string` (from `<string>`) is a full class that manages its own dynamically-growing memory, supports the `+` operator for concatenation and `==` for comparison, and exposes convenient member functions such as `length()`, `substr()` and `find()`. A short demonstration program (see Topic 1.8) declares both a char `name[20]` and a string `name="Asha"`, shows `strlen()` vs `.length()`, and `+` concatenation producing "Asha Patil" — illustrating that `std::string` is safer and more convenient for modern C++ code, while C-strings remain relevant for interfacing with C APIs and legacy code.

Q3. [5 marks] Explain pointers in C++ with syntax, an example program and its output.

A pointer is a variable that stores the memory address of another variable rather than a value directly. It is declared as type `*name;`, given an address using the address-of operator `&` (`p = &x;`), and the pointed-to value is accessed by dereferencing with `*p`. See the worked program under Topic 1.7: `int x=25; int *p=&x; prints x=25, &x and p as the same address, *p as 25, and after *p=50; the value of x itself becomes 50, demonstrating that modifying through a pointer modifies the original variable. Pointers are essential for dynamic memory allocation (new/delete), efficient array handling, and building dynamic data structures such as linked lists.`

Q4. [5 marks] Explain the relationship between arrays and pointers in C++ with an example.

An array name, in most expressions, automatically decays into a pointer to its first element, so `arr` and `&arr[0]` are equivalent, and element access via `arr[i]` is exactly equivalent to `*(arr + i)`. This is demonstrated by declaring `int arr[5] = {10,20,30,40,50}; int *p = arr;` and showing that `arr[i]` and `*(p+i)` print identical values for every index. However, `sizeof(arr)` still reports the whole array's byte size, while `sizeof(p)` reports only the size of one pointer, which is a key distinction tested in exams: once an array is passed to a function, it decays to a pointer and its original size information is lost unless passed separately.

Q5. [5 marks] Explain enumeration types with syntax, rules and an example program.

An enum defines a user-named type made of a set of named integer constants, improving code readability compared with raw "magic numbers". Declared as `enum Day { SUN, MON, ..., SAT };` the first enumerator defaults to 0 and each subsequent one increases by 1 unless given an explicit value (`enum Grade { A=5, B=4, C=3 };`). A plain enum value implicitly converts to int, while a scoped enum class (C++11) does not, which avoids naming clashes and accidental int conversions, and is generally recommended in modern code. The worked program (Topic 1.11) declares `Day today = WED;` prints its integer code (3), and uses it in an if condition to classify weekday vs weekend, showing enums used for control flow in a readable way.

Q6. [5 marks] What is the this pointer? Explain with an example showing its two common uses.

(Also relevant to Unit II, but fundamentally a C++ language feature.) "this" is an implicit pointer available inside every non-static member function, pointing to the specific object on which the function was invoked. Its two most common uses are: (1) resolving a naming clash when a parameter has the same name as a data member, via `this->member = parameter`; and (2) returning `*this` from a member function to allow method chaining, e.g. `obj.add("a").add("b")`; A short illustrative class with a `setXY(int x, int y)` method using `this->x=x`; `this->y=y`; demonstrates use (1).

Q7. [5 marks] Discuss the various fundamental, derived and user-defined data types in C++ with examples.

C++ data types fall into three groups. Fundamental (built-in) types include `int`, `float`, `double`, `char`, `bool` and `void`, each with a typical size and range (see the `sizeof` table under Topic 1.2). Derived types are built from fundamental types: arrays (`int a[10]`); pointers (`int *p`); references (`int &r`); and functions. User-defined types are created by the programmer: `class`, `struct`, `union`, `enum` and typedef-based aliases. This three-way classification is a standard exam question, usually expecting the student to name at least two examples per category and briefly state typical memory size where relevant.

C. Ten-Mark / Long-Answer Questions

Q1. [10 marks] Explain Object-Oriented Programming concepts in detail, contrasting OOP with procedural programming, and illustrate each pillar with a C++ program.

Procedural programming (e.g. plain C) organises a program as a sequence of instructions/functions that operate on shared or global data, which becomes hard to manage as programs grow, since data can be modified from anywhere. Object-Oriented Programming instead organises a program around objects that bundle their own data and the functions that operate on it, giving four core pillars: **Encapsulation** (binding data+functions, hiding internals via private access — demonstrated by an `Account` class that only allows balance changes through a `deposit()` method), **Abstraction** (showing only essential operations while hiding how they are implemented — e.g. calling `area()` without knowing the formula), **Inheritance** (a derived class such as `Dog : public Animal` reuses `Animal`'s members and adds its own, reducing duplicated code), and **Polymorphism** (the same interface behaving differently per type — function overloading such as `add(int,int)` vs `add(double,double)` at compile time, and virtual functions resolved at run time, covered fully in Unit III). A complete answer should include at least one short, compilable program per pillar, and conclude that OOP's benefits are modularity, reusability, easier maintenance and closer mapping to real-world entities, at the cost of a steeper learning curve and sometimes more overhead than minimal procedural code.

Q2. [10 marks] Describe in detail the various fundamental and derived data types available in C++, their sizes, ranges and usage, with example programs.

See the full Topic 1.2 table (type, typical size, range) reproduced from this chapter, plus the `sizeof` demonstration program. A strong answer explains that: `char` typically occupies 1 byte and represents a single character or small integer (-128 to 127); `int` typically occupies 4 bytes for whole numbers; `float` (4 bytes, ~6-7 significant digits) and `double` (8 bytes, ~15 significant digits) represent real numbers with different precision/memory trade-offs; `bool` (1 byte) represents true/false. It should also cover type modifiers (short, long, signed, unsigned) that adjust the range of an existing type, and explicitly note that sizes are implementation-defined — the `sizeof` operator should always be used to confirm actual sizes on a given compiler/platform, as shown in the worked program.

Q3. [10 marks] Explain pointers in C++ comprehensively: declaration, the `&` and `*` operators, pointer arithmetic, pointers and arrays, and dynamic memory allocation with `new/delete`. Support with programs.

A pointer is declared as type `*name`; and given an address with `&` (address-of); the stored address is followed ("dereferenced") with `*`. Pointer arithmetic (`p+1`) advances by `sizeof(type)` bytes, which underlies array traversal: `arr[i]` is defined to behave exactly as `*(arr+i)`, since an array name decays to a pointer to its first element in most expressions (Topic 1.13). Dynamic memory allocation uses `new` to request heap memory at run time (`int *p = new int(99)`;) and `delete` to release it (`delete p`;) — forgetting `delete` causes a memory leak, and using a pointer after `delete` (without setting it to `nullptr`) causes a dangling-pointer bug. A full answer includes the basic pointer program (Topic 1.7), the pointer-array equivalence program (Topic 1.13), and the dynamic allocation example, each with its expected output, plus a short note on common pointer-related bugs (see "Common Mistakes" in this chapter).

Q4. [10 marks] Discuss Strings in C++: C-style strings vs `std::string`, common operations, and write a program to demonstrate string concatenation, length and substring extraction.

C-style strings are null-terminated `char` arrays, manipulated with `<cstring>` functions such as `strlen()`, `strcpy()`, `strcat()` and `strcmp()`; their fixed size and lack of bounds checking make them error-prone. `std::string` (from `<string>`) is the preferred modern approach: it grows dynamically, supports natural operators (`+` for concatenation, `==` for comparison, `[]` for indexing) and member functions such as `.length()`, `.substr(pos,len)`, `.find()` and `.empty()`. The answer should present a program that declares a C-string and a `std::string` side by side, concatenates "Asha" + " Patil" to get "Asha Patil", prints its `.length()` (10) and extracts a `.substr(0,4)` ("Asha"), matching the worked example in Topic 1.8, and should discuss when C-strings are still necessary (interfacing with C libraries, embedded/legacy code).

Q5. [10 marks] Explain the concept of inheritance's prerequisite building blocks covered in this chapter — classes, references and the const qualifier — and how they combine to support safe, efficient function design. Illustrate with a program using const reference parameters.

A class groups data and behaviour (Topic 1.16); a reference is a permanent alias for a variable that avoids copying and allows a function to modify the caller's variable directly (Topic 1.10); const prevents unwanted modification of a variable, pointer, or parameter (Topic 1.9). Combining them, a function parameter declared as `const ClassName &obj` gives the best of both worlds: no copy overhead for a potentially large object (because it is passed by reference), while the const guarantees the function cannot accidentally modify the caller's object — a pattern used pervasively in Unit II's copy constructors (Topic 2.9) and operator overloads (Topic 2.13). A demonstration program can show a function `void printStudent(const Student &s)` that displays a Student's data without copying or modifying it, explicitly contrasting it with a pass-by-value version (extra copy made) and a non-const reference version (caller's object could accidentally be modified).

D. Important Programming Questions

PRACTICE: WRITE THE FULL PROGRAM

- Write a C++ program to swap two numbers using pointers (without a temporary variable using arithmetic, and also using a temporary variable).
- Write a C++ program to find the largest and smallest element in an array using pointer arithmetic to traverse the array.
- Write a C++ program that declares a 3x3 matrix, reads it from the user, and prints its transpose.
- Write a C++ program to count vowels and consonants in a string entered using `std::string`.
- Write a C++ program using an enum to represent the days of the week, and print "Weekday" or "Weekend" for a day entered by the user.

E. Output Prediction Questions

Q1. Predict the output:

```
#include <iostream>
using namespace std;

int main() {
    int a = 10;
    int *p = &a;
    *p += 5;
    cout << a << " " << *p;
    return 0;
}
```

ANSWER

15 15

Explanation: `a` and `*p` refer to the same memory; adding 5 through the pointer changes `a` itself, so both print 15.

Q2. Predict the output:

```
#include <iostream>
using namespace std;

int main() {
    int arr[4] = {2, 4, 6, 8};
    int *p = arr;
    cout << *(p + 2) << " " << p[3];
    return 0;
}
```

ANSWER

6 8

Explanation: `*(p+2)` is the element at index 2 (value 6); `p[3]` is the element at index 3 (value 8) — both notations are equivalent to array indexing.

Q3. Predict the output:

```
#include <iostream>
using namespace std;

int main() {
    const int x = 5;
    int y = 10;
    const int *p = &y;
    p = &x;        // allowed: p itself can be re-pointed
    cout << *p;
    return 0;
}
```

ANSWER

5

Explanation: p is a "pointer to const int", meaning *p cannot be modified through p, but p itself can be re-pointed to a different const or non-const int; after reassignment it points to x, so *p prints 5.

Q4. Predict the output:

```
#include <iostream>
using namespace std;

int main() {
    bool a = 7, b = 0;
    cout << boolalpha << a << " " << b;
    return 0;
}
```

ANSWER

true false

Explanation: Any non-zero value converts to true (7 -> true); 0 converts to false. boolalpha prints these as words instead of 1/0.

F. Find-the-Error Questions

Q1. Identify and correct the error:

```
#include <iostream>
using namespace std;

int main() {
    int x = 10;
    int &ref;        // ERROR
    ref = x;
    cout << ref;
    return 0;
}
```

EXPLANATION

A reference must be initialised at the point of declaration (it cannot exist without referring to something). Fix: `int &ref; = x;`

Q2. Identify and correct the error:

```
#include <iostream>
using namespace std;

int main() {
    const int LIMIT = 100;
    LIMIT = 200;    // ERROR
    cout << LIMIT;
    return 0;
}
```

EXPLANATION

LIMIT is declared const, so it cannot be reassigned after initialisation. Any attempt to modify a const variable is a compile-time error.

Q3. Identify and correct the error:

```
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {1,2,3,4,5};
    for (int i = 0; i <= 5; i++)    // ERROR (off-by-one)
        cout << arr[i] << " ";
    return 0;
}
```

EXPLANATION

Valid indices are 0 to 4; the condition `i <= 5` reads `arr[5]`, which is out of bounds (undefined behaviour). Fix: use `i < 5`.

Q4. Identify and correct the error:

```
#include <iostream>
using namespace std;

int main() {
    int *p;
    cout << *p;    // ERROR
    return 0;
}
```

EXPLANATION

`p` is declared but never assigned a valid address (a wild pointer); dereferencing it is undefined behaviour and may crash the program. Fix: initialise `p` (e.g. `int x; p = &x;`) before dereferencing.

G. Multiple Choice Questions (with explanations)

1. Which of the following is NOT one of the four pillars of OOP?

(A) Encapsulation (B) Compilation ✓ (C) Inheritance (D) Polymorphism

Answer: (B). "Compilation" is a build process, not an OOP principle. The four pillars are Encapsulation, Abstraction, Inheritance and Polymorphism.

2. What is the default access specifier for members of a class (not struct) in C++?

(A) public (B) protected (C) private ✓ (D) friend

Answer: (C). Class members are private by default in C++; struct members default to public — this is the only structural difference between class and struct.

3. Which operator is used to get the address of a variable?

(A) * (B) & ✓ (C) -> (D) ::

*Answer: (B). & is the address-of operator; * is the dereference operator used to access the value at an address.*

4. A reference variable, once initialised, can:

(A) Be made to refer to a different variable (B) Never be made to refer to a different variable ✓ (C) Be null (D) Be left uninitialised
Answer: (B). A reference is permanently bound to the variable it was initialised with; it cannot be reseated, be null, or be left uninitialised.

5. What does the following print? `cout << sizeof(char);`

(A) 2 (B) 4 (C) 1 ✓ (D) 8

Answer: (C). A char occupies 1 byte on virtually all C++ implementations.

6. Which of these is the correct syntax for a pointer to a constant integer?

(A) `int *const p;` (B) `const int *p;` ✓ (C) `const int const p;` (D) `int const *const p;`

*Answer: (B). "const int *p;" means the pointed-to value cannot change through p (pointer to const); "int *const p;" instead means p itself cannot be re-pointed (a constant pointer).*

7. An enum's first enumerator, if not given an explicit value, defaults to:

(A) 1 (B) -1 (C) 0 ✓ (D) undefined

Answer: (C). Unless given an explicit value, the first enumerator of an enum defaults to 0, and each subsequent one increases by 1.

8. Which header must be included to use `std::string`?

(A) (B) (C) ✓ (D)

Answer: (C). `std::string` is defined in the header; is for C-style string functions like `strlen()` and `strcpy()`.

9. What does an array name "decay" into in most expressions?

(A) A reference to the array (B) A pointer to its first element ✓ (C) A copy of the whole array (D) An integer representing its size

Answer: (B). In most expressions, an array name decays into a pointer to its first element, which is why `arr[i]` and `*(arr+i)` are equivalent.

10. Which qualifier tells the compiler a variable may change outside the normal program flow (e.g. hardware)?

(A) `const` (B) `static` (C) `volatile` ✓ (D) `extern`

Answer: (C). `volatile` prevents the compiler from optimising away repeated reads of a variable whose value can change unexpectedly (e.g. from an interrupt handler or hardware register).

H. Viva-Voce Questions

BE READY TO ANSWER VERBALLY, IN YOUR OWN WORDS

- What is the difference between a class and a structure in C++?
- Why is C++ called a "hybrid" language?
- What happens if you forget to initialise a pointer before dereferencing it?
- Can a reference be reseated after initialisation? Why or why not?
- What is the difference between pass-by-value, pass-by-reference and pass-by-pointer?
- Why does `std::string` generally remain preferred over char arrays in modern C++ code?
- What is the difference between a pointer to a constant and a constant pointer?
- How does `sizeof` differ for an array versus a pointer to the same data?
- What is the practical difference between enum and enum class?
- What does the `volatile` keyword prevent the compiler from doing?
- What is a typedef used for, and how does using differ from it?
- Why must a `const` variable be initialised at the point of declaration?
- What is the difference between declaration and definition?
- What are escape sequences, and why are they needed?
- Give one real-world analogy each for encapsulation and abstraction.

I. Chapter 1 — Revision Notes (Must-Remember Points)

TOP 10 EXAM-CRITICAL POINTS

- OOP's four pillars: Encapsulation, Abstraction, Inheritance, Polymorphism (EAIP).

- Fundamental types: int, float, double, char, bool, void; typical sizes 4,4,8,1,1 bytes (platform-dependent — always confirm with sizeof).
- Pointer: stores an address, needs * to dereference, can be reseated, can be null, supports arithmetic.
- Reference: an alias, no dereference needed, cannot be reseated, cannot be null, must be initialised at declaration.
- const protects a value from modification; "const int *p" protects the pointee, "int *const p" protects the pointer itself.
- An array name decays to a pointer to its first element: $\text{arr}[i] \equiv *(\text{arr} + i)$.
- std::string is safer and more convenient than C-style char arrays for almost all modern code.
- enum values default to 0,1,2...; enum class avoids implicit int conversion and naming clashes.
- typedef / using create a new NAME for an existing type, not a new type.
- volatile stops the compiler from optimising away repeated reads of a value that can change externally.